

Foundations of Data Science for Biologists

What is (and why learn) Unix?

BIO 724D

2026-JAN-08

Instructors: Greg Wray and Kayla Wilhoit

Unix was started by rebels

Computing world in 1960s

Computers were accessible to *very* few individuals

Hardware and software was proprietary, quirky, and constantly changing

Computing was enormously expensive and exclusive, required specialized training



Unix was started by rebels

Computing world in 1960s

Computers were accessible to *very* few individuals

Hardware and software was proprietary, quirky, and constantly changing

Computing was enormously expensive and exclusive, required specialized training

A few scientists at Bell Labs in the late 1960s had a different vision

A flexible, agile, stable, collaborative computing environment

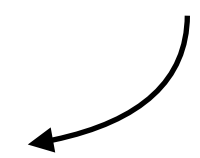
Independent of proprietary hardware and software

*They succeeded!
And changed the world!*

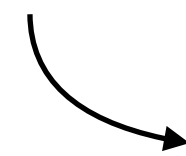
Billy Joel



problem



solution



Public domain

What is Unix?

Reaction to proprietary, complicated, expensive, inefficient computing of 1960s

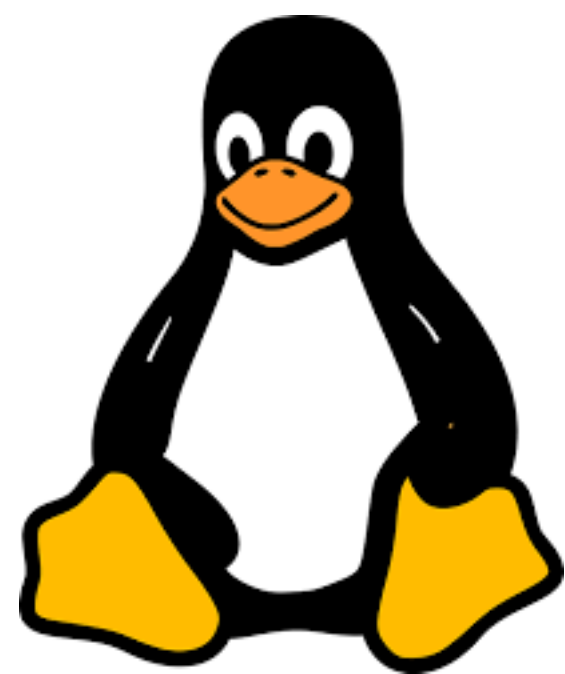
Development began in 1969 at Bell Labs; most features present by mid-1970s

Became hugely popular during the 1980s, first OS freely available in academia

Source (directly or indirectly) of all modern operating systems (OS)

Introduced many, massively important innovations

For the past 40+ years, Unix and its derivatives have dominated the computing world



Linux



Mac



Windows

etc.

everything else

Why was Unix so successful?

Successfully *implemented* important* concepts that were ineffective in previous OS

- Hierarchical file system

- Multi-process

- Multi-user

Successfully *introduced* several important* innovations

- Pipes

- Regular expressions

- Hardware abstraction

- Portability

- Self-sufficiency

- Modular design

* paradigm-shifting, mind-boggling, literally civilization-changing

There are reasons you might hesitate to learn Unix

Learning curve can seem **steep** compared with familiar window-based interfaces

Command-line interface can initially feel **awkward** and **unituitive**



```
Code — -zsh — 119x33
gwrap@AUDS-DP4X9W6MFJ Code % pwd
/Users/gwrap/Documents/Projects_home/Database_project/Code
gwrap@AUDS-DP4X9W6MFJ Code % ls -la
total 40680
drwx----- 19 gwrap staff      608 Jan  6 08:28 .
drwx----- 14 gwrap staff      448 Jan  6 08:28 ..
-rw-r--r--@ 1 gwrap staff     6148 Jan  5 10:53 .DS_Store
drwxr-xr-x  7 gwrap staff      224 Jan  5 10:19 .ipynb_checkpoints
-rwx-----@ 1 gwrap staff    90488 Jan  6 08:28 Build_savvyBird.ipynb
-rw-r--r--@ 1 gwrap staff   116051 Jan  5 19:59 Cleaning_data.pages
-rw-r--r--@ 1 gwrap staff    4114 Jan  5 09:59 bioregions.csv
-rw-r--r--@ 1 gwrap staff   19844 Jan  5 20:34 countries.csv
-rw-r--r--@ 1 gwrap staff   57112 Jan  5 09:59 families.csv
-rw-r--r--@ 1 gwrap staff  100378 Jan  5 20:51 genera.csv
-rw-r--r--@ 1 gwrap staff  110875 Jan  5 10:00 locations.csv
-rw-r--r--@ 1 gwrap staff 2803044 Jan  5 10:00 observations.csv
-rw-r--r--@ 1 gwrap staff    4374 Jan  5 10:00 orders.csv
-rw-r--r--@ 1 gwrap staff    1836 Jan  5 19:22 otest.csv
-rw-r--r--@ 1 gwrap staff    1831 Jan  5 19:38 otest_2.csv
-rw-r--r--  1 gwrap staff 12857344 Jan  5 20:51 savvybird.db
-rw-r--r--  1 gwrap staff  3626075 Jan  5 20:52 savvybird.db.wal
-rw-r--r--@ 1 gwrap staff   265197 Jan  5 10:01 species.csv
-rw-r--r--@ 1 gwrap staff    23346 Jan  5 10:01 trips.csv
gwrap@AUDS-DP4X9W6MFJ Code %
```


Reasons to love Unix (and why it continues to dominate modern computing)

good investment

Widely adopted

Future-proof

Reproducible

capabilities

More control

Scalable

Automation

Workflows

Software

future you

Professional

Terms and disambiguation: “parts” of UNIX

Kernel: core of the OS

Interacts directly with hardware

Shell: user - kernel interface

Passes instructions to the kernel

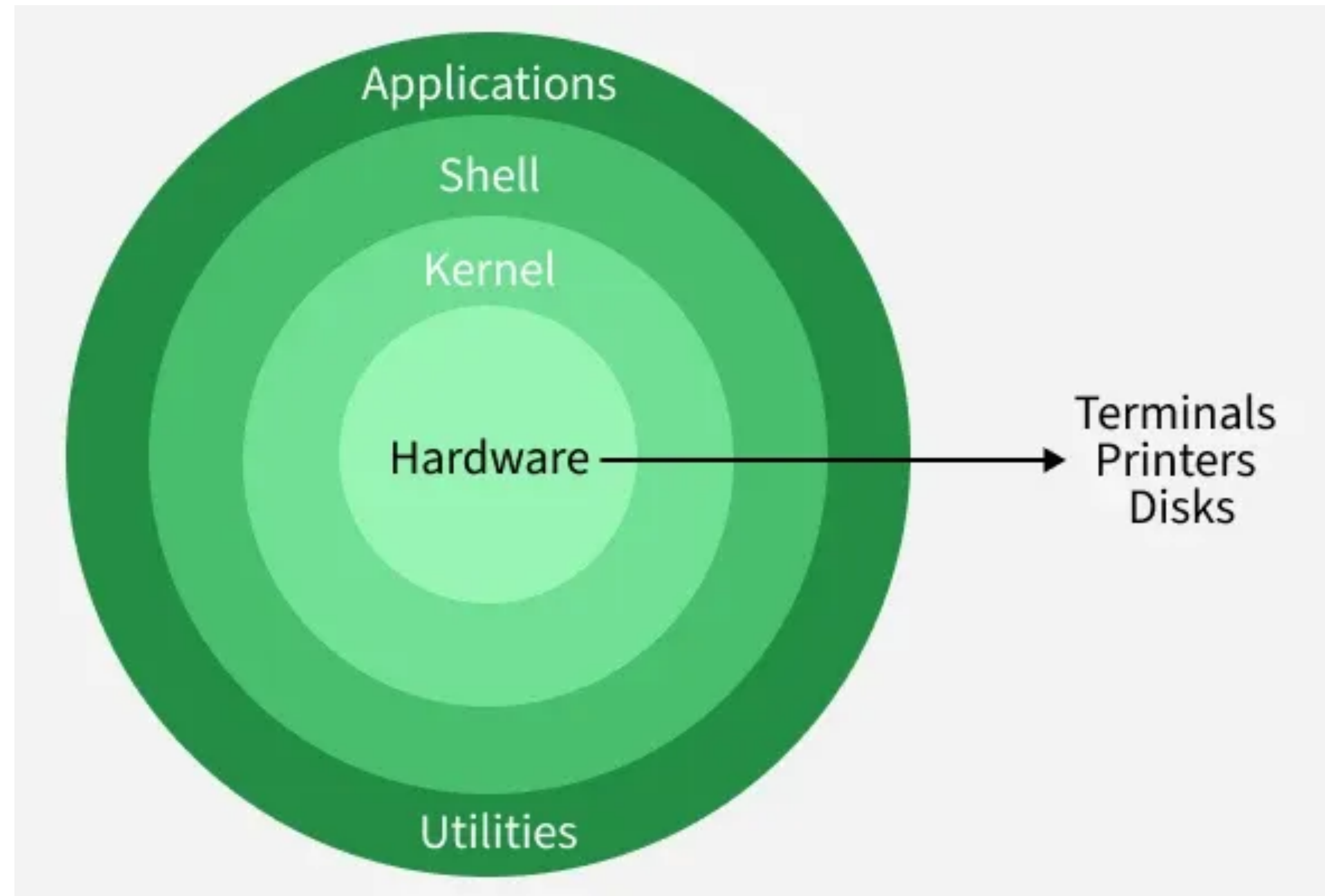
Reports back from the kernel

Terminal: how we interact with the shell

AKA the command line

Tools: mostly standard set of >60 utilities

Take the place of clicking on icons



Reference section

Why was Unix so successful?

Successfully *implemented* important* concepts that were ineffective in previous OS

Hierarchical file system: infinite levels and namespace, easily managed

Multi-process: simple, effective, scalable process management; start and ignore

Multi-user: simple, efficient, transparent sharing of hardware resources

Successfully *introduced* several important* innovations

Pipes: support for passing results from one program to the next

Regular expressions: integrated into many utilities

Hardware abstraction: complications of hardware interaction hidden from users

Portability: same commands and programs run on different hardware installations

Self-sufficiency: everything needed to support/update is part of the OS

Modular design: updating one part doesn't upset other parts

* paradigm-shifting, mind-boggling, civilization-changing

Reasons to love Unix (and why it continues to dominate modern computing)

Widely adopted: the most universal computing environment ever

Future-proof: the most stable environment for writing programs, creating work flows

Reproducible: maximizes reproducibility; see above two reasons

More control: achieve much more control over your computer and files

Scalable: the only way to access most high-resource, cluster computing

Automation: schedule tasks, automate data-logging and -processing

Workflows: link programs into pipes to achieve immensely powerful capabilities

Software: many useful programs are only accessible through Unix

Professional: required skill set in many academic and industry positions

Terms and disambiguation: “parts” of UNIX

Kernel: core of the OS

- Manages programs and requests from the command line

- Manages files, memory, remote connections, users, and security

Shell: interface between the user and the kernel

- Interprets commands and passes instructions to the kernel

- Reports back from the kernel on the status and results of commands

Terminal: mechanism for interacting with the shell (the **command line**)

- Originally, a physical device connected to the computer

- Today we use a graphical interface that emulates a physical terminal

Tools: mostly standard set of >60 programs provided with the shell

- Most originally developed in the 1970s, expanded slightly since then

Terms and disambiguation: “flavors” of *NIX

UNIX: proprietary; “true UNIX”

- Developed by Bell Labs; licensed to industry (expensive) and academia (free)

- Currently owned and maintained as a standard by The Open Group

Linux: open-source; a complete re-write based on the UNIX specification

- Kernel developed by Linus Torvalds

- Almost everything else developed by GNU

- Many commercial distributions (“distros”): Ubuntu, Centos, Debian, RedHat, etc.

MacOS: open-source and proprietary

- Darwin kernel and tools based on Berkeley BSD UNIX are open-source

- GUI is proprietary

Phylogeny of all OS: https://eylenburg.github.io/os_familytree.htm

Terms and disambiguation: shells

cshtcsh: C shell

Early shell with C-like syntax; written by a graduate student at UC Berkeley

Part of BSD Unix for academic settings

bash: Bourne again shell

Written in 1979; default on our VMs, most common shell in Linux distros

zsh: Z shell

Written in 1990, also by a graduate student; default on MacOS and some Linux distros

PowerShell: does not interface with true *NIX; runs in Windows OS

Understands standard UNIX commands and tools

POSIX: a standard for shells

Sticking to POSIX syntax, tools, and options makes code portable among shells

